TUGAS BESAR IF2224
MILESTONE 1
LEXICAL ANALYSIS

Kelompok std_abs

Ega Luthfi Rais	13524115
Salman Faiz Assidqi	13524134
Ahmad Fauzan Putra	13524141
Leonardus Brain Fatolosja	13524146

Institut Teknologi Bandung

2026

DAFTAR ISI

DAFTAR ISI **2**
1.1 Determine Finite Automata (DFA) 3
1.2 Compiler dan Interpreter 3
1.3 Lexical Analysis 4
PERANCANGAN & IMPLEMENTASI **6**
2.1 Rancangan Determine Finite Automata (DFA) 6
2.2 Implementasi Determine Finite Automata 8
3.1 Pengujian 1: 9
3.2 Pengujian 2: 9
3.3 Pengujian 3: 9
3.4 Pengujian 4: 9
3.5 Pengujian 5: 9
4.1 Kesimpulan 10
4.2 Saran 10
LAMPIRAN **11**
REFERENSI **12**

BAB 1 TEORI SINGKAT

1.1 Determine Finite Automata (DFA)

DFA adalah sebuah konsep (model) di dalam *computer science* yang digunakan untuk merepresentasikan *state* atau keadaan. DFA merupakan deterministik karena untuk setiap keadaan (state), DFA mempunyai semua kemungkinan input (transisi) yang mengarah ke *state* lain atau *state* itu sendiri dan untuk setiap input, hanya ada satu tujuan atau transisi sehingga tidak adanya ambiguitas di dalam DFA. Dibandingkan dengan NFA atau Regular Expression, DFA lebih mudah untuk diimplementasikan dalam program dan lebih mudah dalam proses komputasinya.

Secara formal, DFA didefinisikan sebagai 5-tupel, yaitu antara lain:

a. Q, sebagai himpunan *state*

b. Σ, sebagai himpunan solusi input

c. δ, sebagai fungsi transisi

d. q0, sebagai *state* awal

e. F, sebagai himpunan state akhir (*final state*)

DFA menerima sebuah *string*, yaitu masukan yang akan diproses oleh DFA. DFA akan mengecek tiap input yang masuk dari string tersebut dan menjalankan fungsi transisi sehingga akan ada perpindahan *state*. Ketika *string* berakhir, posisi keadaan terakhir akan menentukan apakah DFA menerima *string* tersebut atau tidak. Jika keadaan (*state*) terakhir berada di *final state*, maka DFA menerima *string* tersebut, sedangkan jika *state* tidak berakhir di *final state*, maka DFA menolak *string* tersebut.

DFA memiliki peranan penting pada dunia komputer terkhususnya. Pada lingkup keamanan jaringan, DFA bisa dipakai untuk mendeteksi paket-paket data internet dan filtering. Selain itu, DFA juga dipakai pada compiler dan interpreter pada tahap Lexical Analysis. Pada Lexical Analysis, DFA mendeteksi tiap input yang dimasukkan dan menghasilkan token-token yang sesuai. DFA juga dikenal karena komputasinya yang efisien dan cepat karena menggunakan pengambilan keputusan yang deterministik dan selalu sama untuk satu transisi sehingga algoritma yang dibuat tidak berat komputasinya.

1.2 Compiler dan Interpreter

Compiler dan Interpreter adalah dua jenis *software* yang berfungsi untuk mengubah teks (*source code*) menjadi bentuk yang bisa dijalankan oleh perangkat keras/komputer (*machine code*).

Keduanya memiliki tujuan yang sama, yaitu menjalankan instruksi yang dituliskan oleh manusia dalam bahasa yang tingkatnya lebih tinggi (semakin tinggi, semakin dekat dengan bahasa manusia). Perbedaan antara keduanya berada di mekanisme atau pemrosesan nya.

Compiler akan menerjemahkan seluruh *source code* ke dalam sebuah bahasa tujuan (file objek, executable, dll) sebelum program dijalankan. Karena translasi dilakukan di awal, kesalahan sintaks pada program akan diketahui sebelum program dijalankan, bahkan program tidak dapat dijalankan jika sintaks pada *source code* masih belum sesuai. Compiler sendiri terdiri dari beberapa tahap, yakni:

a. Lexical Analysis
b. Syntax Analysis / Parsing
c. Semantic Analysis
d. Intermediate Code Generation
e. Code Optimization
f. Code Generation

Sebaliknya, interpreter mengeksekusi program dengan cara membaca, menerjemahkan, dan menjalankan instruksi secara langsung pada saat program dieksekusi/*source code* dibaca. Pada pendekatan ini, program tidak selalu membuat file hasil dengan bahasa tujuan. Kesalahan sintaks pun akan langsung diketahui ketika program dijalankan. Secara garis besar, Interpreter biasanya lebih fleksibel, namun lebih lambat dibandingkan dengan compiler. Interpreter sendiri terdiri dari beberapa tahap, yakni:

a. Lexical Analysis
b. Syntax Analysis / Parsing
c. Semantic Analysis
d. Intermediate Code Generation
e. Execution

1.3 Lexical Analysis

Lexical analysis merupakan tahap awal dalam proses penerjemahan *source code*/program, baik pada compiler maupun interpreter. Tahap ini bertugas membaca tiap karakter dari kode sumber, kemudian mengelompokkannya menjadi unit-unit bermakna yang disebut token. Proses ini dilakukan sebelum *Syntax Analysis*, sehingga lexical analysis berperan sebagai penghubung antara teks mentah dengan struktur formal yang akan diproses pada tahap berikutnya.

Token adalah representasi kategoris dari bagian-bagian penting dalam program, seperti keyword, identifier, operator, literal, dan tanda baca. Selain menghasilkan token, lexical analysis juga berfungsi untuk mengabaikan elemen-elemen yang tidak berpengaruh langsung terhadap struktur

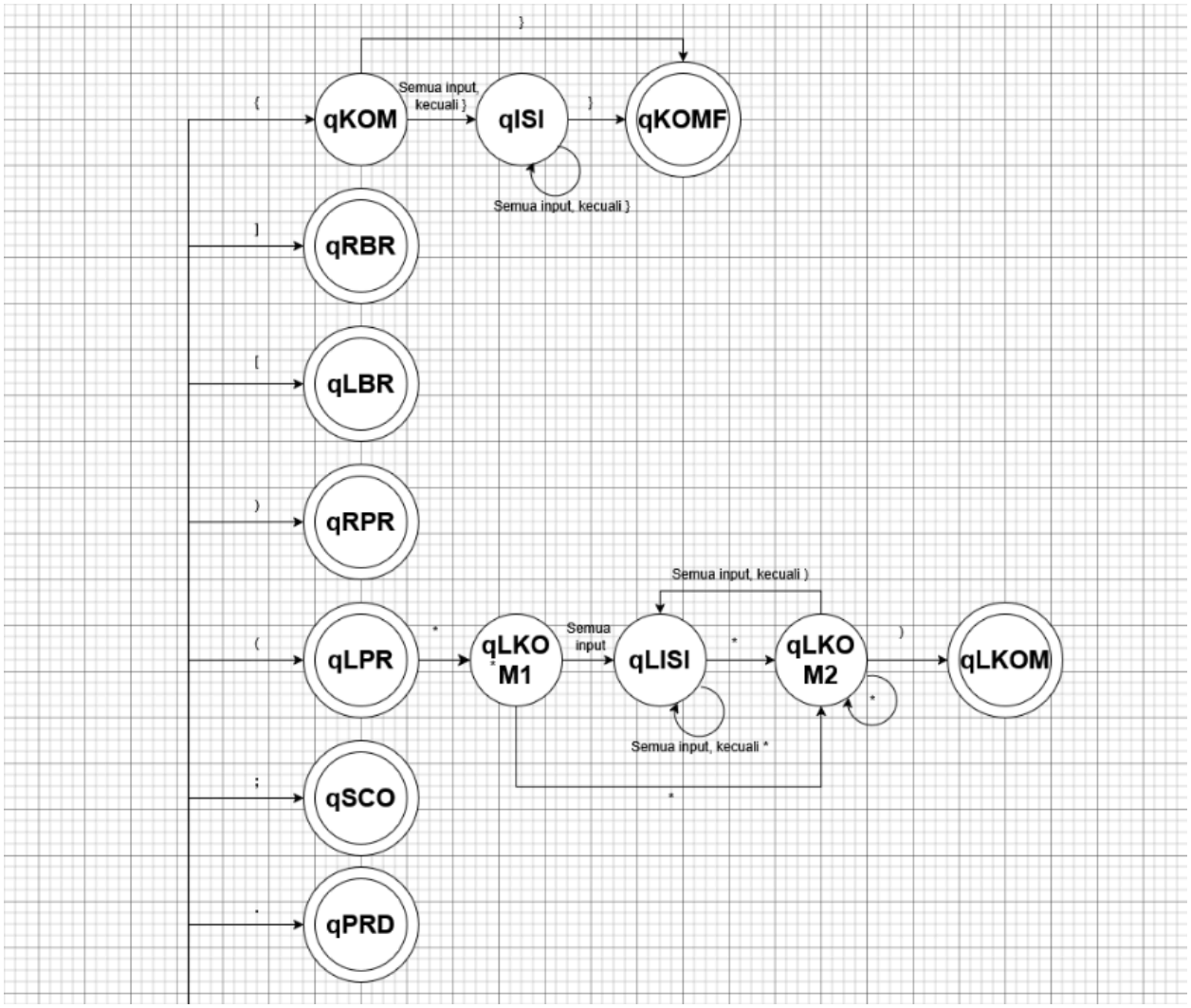
sintaksis, seperti spasi, tab, dan komentar. Pada tahap ini pula, kesalahan leksikal dapat dideteksi, misalnya adanya simbol yang tidak dikenali atau bentuk token yang tidak valid.

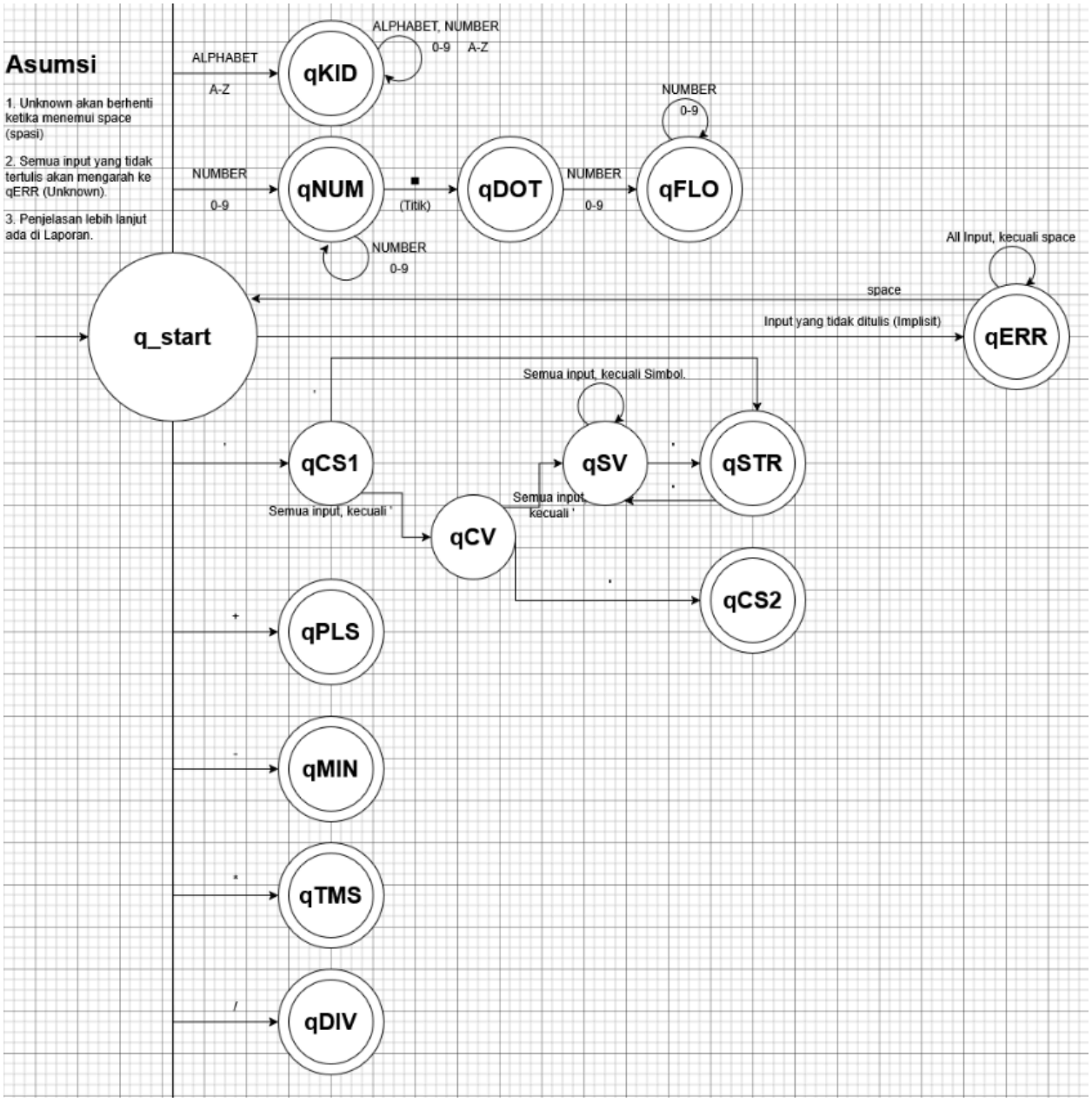
Dalam implementasinya, lexical analyzer umumnya dibangun berdasarkan konsep regular expression dan finite automata. Setiap pola token didefinisikan dalam bentuk aturan tertentu, lalu direalisasikan ke dalam automata yang dapat memproses input secara sistematis. Oleh karena itu, keterkaitan antara DFA dan lexical analysis sangat erat. DFA menyediakan mekanisme formal yang efisien untuk mengenali pola-pola token dari aliran karakter input.

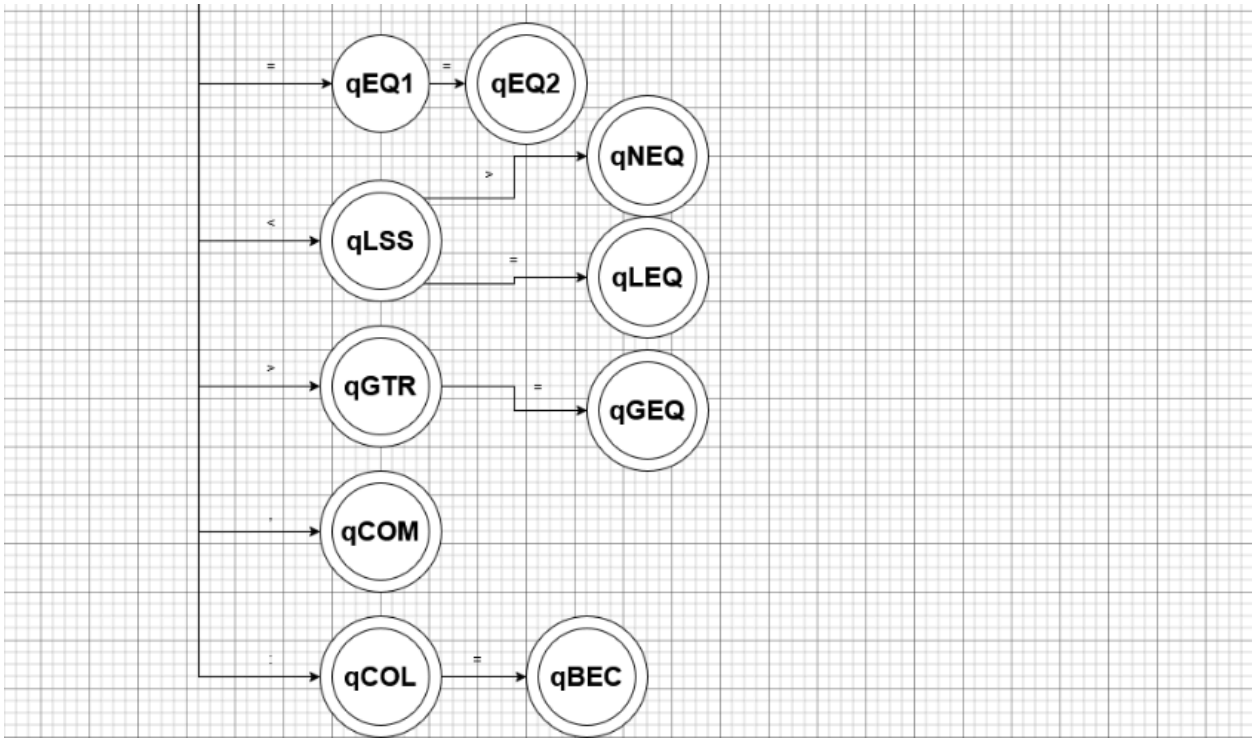
BAB 2
PERANCANGAN & IMPLEMENTASI

2.1 Rancangan Determine Finite Automata (DFA)

Berikut adalah diagram transisi dari DFA:




Asumsi
1. Unknown akan berhenti ketika menemui space (spasi)
2. Semua input yang tidak tertulis akan mengarah ke qERR (Unknown).
3. Penjelasan lebih lanjut ada di Laporan.
ALPHABET
A-Z
qKID
ALPHABET, NUMBER
0-9 A-Z
NUMBER
0-9
qNUM
.
(Titik)
qDOT
NUMBER
0-9
qFLO
NUMBER
0-9
q_start
All Input, kecuali space
space
Input yang tidak ditulis (Implisit)
qERR
Semua input, kecuali Simbol.
'
qCS1
Semua input, kecuali '
qCV
Semua input, kecuali '
qSV
qSTR
qCS2
+
qPLS
-
qMIN
*
qTMS
/
qDIV



DFA di atas memiliki himpunan state yang memiliki makna sebagai berikut.

a. q_start : State awal menandakan dimulainya suatu DFA dan berarti bahwa DFA belum menerima *string* atau input apa pun. Selain itu, state ini dapat diartikan bahwa DFA sudah selesai membaca suatu input sebelumnya (Contohnya: setelah membaca comma, maka DFA akan "return" comma dan kembali ke state awal menandakan bahwa string tersebut berhenti di sana).
b. q_KID : State yang menerima alphabet sebagai input awalnya. Makna dari state ini adalah isi dari state ini bisa mengandung dua kemungkinan, yaitu keyword (Token-token yang berisi alphabet, seperti array, while, for, dll.) dan ident (kombinasi token-token tidak berada di daftar keyword)
c. q_NUM : State yang menerima angka sebagai input awalnya. Maknanya adalah state ini mengandung kondisi di mana kondisi input berisi dua kemungkinan, yaitu angka *integer* (tidak ada titik) atau *float* (ada titik).
d. q_DOT : State yang menerima titik setelah menerima angka. Maknanya adalah state ini mengandung kondisi di mana input akan berubah menjadi *float*. Jika input selanjutnya bukan angka, maka akan masuk ke state q_ERR (error/unknown).
e. q_FLO : State yang menerima angka setelah titik yang sebelumnya menerima angka (q_DOT). Maknanya adalah state ini mengandung kondisi di mana state berisi string yang merupakan *float*.
f. q_CS1 : State yang menerima ' sebagai input awalnya. Maknanya adalah state ini mengandung dua kondisi, yaitu input yang akan menjadi charcon atau string.

g. q_CSV : State yang menerima satu karakter setelah menerima ‘ . Maknanya adalah state ini mengandung dua kondisi, yaitu charcon dan string sekaligus sudah tidak akan menerima string kosong.
h. q_CS2 : State yang menerima ‘ setelah 1 karakter. Maknanya adalah input yang dimasukkan telah berhasil menjadi charcon.
i. q_SV : State yang menerima setidaknya 2 character setelah ‘. Maknanya adalah input sudah dipastikan akan menjadi string, tetapi belum lengkap.
j. q_STR : State yang sudah menerima string ‘ setelah lebih dari 1 karakter. Maknanya adalah input sudah lengkap dan diterima sebagai string.
k. q_PLS : State yang menerima + sebagai input awal. Maknanya adalah input sudah lengkap dan diterima sebagai plus.
l. q_MIN : State yang menerima - sebagai input awal. Maknanya adalah input sudah lengkap dan diterima sebagai minus.
m. q_TMS : State yang menerima * sebagai input awal. Maknanya adalah input sudah lengkap dan diterima sebagai times.
n. q_DIV : State yang menerima / sebagai input awal. Maknanya adalah input sudah lengkap dan diterima sebagai idiv.
o. q_EQ1 : State yang menerima = sebagai input awal. Maknanya adalah input akan mengandung kondisi di mana input akan berubah menjadi == (equal).
p. q_EQ2 : State yang menerima = setelah =. Maknanya adalah input sudah lengkap dan diterima sebagai eql (equal).
q. q_LSS : State yang menerima < sebagai input awal. Maknanya adalah state mengandung tiga kondisi, yaitu sudah lengkap dan diterima sebagai lss (less than), input dari state akan berubah menjadi neq (not equal), dan leq (less than or equal).
r. q_GTR : State yang menerima > sebagai input awal dan bermakna bahwa state mengandung dua kondisi, yaitu input sudah lengkap dan diterima sebagai gtr (greater than) dan dapat berubah menjadi geq (greater or equal).
s. q_geq : State yang menerima = setelah > dan bermakna bahwa input sudah lengkap dan akan diteirma sebagai geq (greater or equal).
t. q_com : State yang menerima , sebagai input awal. Maknanya adalah input sudah lengkap dan diterima sebagai comma.
u. q_col : State yang menerima : sebagai input awal. Maknanya adalah state mengandung dua kondisi, yaitu input sudah lengkap dan diteirma sebagai colon)
v. q_bec : State yang menerima = setelah input : . Maknanya adalah input sudah lengkap dan diterima sebagai becomes.
w. q_prd : State yang menerima . sebagai input awal . Maknanya adalah input sudah lengkap dan diterima sebagai period. Bedanya dengan q_dot adalah q_prd tidak menerima angka sebelumnya.
x. q_sco : State yang menerima ; sebagai input awal . Maknanya adalah input sudah lengkap dan diterima sebagai semicolon.

y. q_lpr : State yang menerima (sebagai input awal . Maknanya adalah state mengandung dua kondisi, yakni input sudah lengkap dan akan diterima sebagai lparent atau dapat menjadi komentar.
z. qLKOM1 : State yang menerima * setelah input awal (. Makananya adalah state mengandung input yang akan berubah menjadi komentar.
aa. qLISI : State yang menerima semua input setelah * atau setelah input yang sebelumnya *. State ini juga akan mengulang ke dirinya terus jika input selanjutnya bukan *. Maknanya adalah kondisi berada di dalam isi komentar.
bb. qLKOM2 : State yang menerima * setelah input yang lain pada state qLISI atau qLKOM1. Maknanya adalah input berkemungkinan berakhir dengan * sebagai string ke-2 dari terakhir. Akan tetapi, string belum tentu berakhir karena masih bisa menerima input selain) dan akan kembali ke state qLISI.
cc. qLKOM : State yang menerima) setelah semua input dari state sebelumnya (qLKOM2). Maknanya adalah input sudah lengkap dan akan diterima sebagai komentar.
dd. qRPR : State yang menerima) sebagai input awal . Maknanya adalah input sudah lengkap dan diterima sebagai rparent.
ee. qLBR : State yang menerima [sebagai input awal . Maknanya adalah input sudah lengkap dan diterima sebagai lbrack.
ff. qRBR : State yang menerima] sebagai input awal. Maknanya adalah input sudah lengkap dan diterima sebagai rbrack.
gg. qKOM : State yang menerima { sebagai input awal . Maknanya adalah input akan menjadi komentar dan jika tidak, maka akan menghasilkan error.
hh. qISI : State yang menerima semua input (kecuali }) setelah input { . Maknanya adalah input berisi sebuah komentar yang berisi setidaknya ada satu input.
ii. qKOMF : State yang menerima } baik setelah input { maupun setelah input isi. Maknanya adalah input sudah lengkap dan akan diterima sebagai komentar (comment)
jj. qERR : State yang menerima input di luar dari transisi yang ditulis dan bermakna bahwa state ini berisi input-input yang tidak dikenali oleh program. Hasilnya adalah sebuah unknown dan **asumsi dari program ini adalah bahwa error/unknown akan berhenti ketika menemui spasi** sehingga selagi belum ada spasi, maka input akan dianggap error/unknown.

2.2 Implementasi Determine Finite Automata

Implementasi menggunakan bahasa pemrograman C++ dengan memanfaatkan konsep sebagai berikut.

BAB 3 PENGUJIAN

3.1 Pengujian 1:

3.2 Pengujian 2:

3.3 Pengujian 3:

3.4 Pengujian 4:

3.5 Pengujian 5:

BAB 4 KESIMPULAN DAN SARAN

4.1 Kesimpulan

4.2 Saran

LAMPIRAN

Link Repository *Github* : https://github.com/road2qnt/ABS-Tubes-IF2224-2026

Link Workspace DFA : https://drive.google.com/file/d/1cVgeMkpIRGY_1Gnye5U6-DMTh6jryJCi/view?usp=sharing

Pembagian Tugas

Nama	Pembagian Tugas	Kontribusi
Ega Luthfi Rais	Project Manager, Inisiasi GitHub, Design struktur gitHub, Assist Design dan Implementasi DFA	25%
Salman Faiz Assidqi		25%
Ahmad Fauzan Putra	Implementasi DFA (token.h dan token.cpp)	25%
Leonardus Brain Fatolosja	Pembuatan DFA, Laporan (Teori singkat dan Penjelasan DFA)	25%

REFERENSI

https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm
https://medium.com/the-modern-scientist/deterministic-finite-automaton-a-fundamental-model-of-computation-330fb65b7c3f
https://www.sciencedirect.com/topics/computer-science/deterministic-finite-automaton
https://www.geeksforgeeks.org/compiler-design/difference-between-compiler-and-interpreter/
https://www.geeksforgeeks.org/compiler-design/phases-of-a-compiler/
https://www.geeksforgeeks.org/compiler-design/token-patterns-and-lexems/